intermediate values. This is perhaps the most simple, yet effective method of debugging (arguably after Rubber Duck debugging). In fact, it is the favourite debugging tool of some famous programmers, as we will soon see.

Some of the major uses of *printf* debugging are:

- To ensure that the program control reaches a particular block and statement
- To ensure that a loop control variable goes in the right order
- To ensure that a pointer isn't NULL before dereferencing it
- To ensure that function calls, especially recursion, get executed in the right order with the right parameters
- To check the return values of library calls in order to identify improper use of the API

The first point on the above list is really important in solving many logical errors, which may puzzle us without leaving any visible error messages. For example, if the file handling code in the following snippet doesn't seem to work as intended, we may find a thousand reasons for this, including permission issues, file system issues and mishandling of the API:

```
if (condition) {
    // multiple lines of complicated file handling code
}
```

But a simple *printf ("I'm here!\n")* inside the *if* block will let us know whether we will ever reach there, revealing the issues with the *if* condition and the previous code that prepares it.

Note: Don't forget to append your *printf* message with a newline character or you won't get the output before the program crashes. This is because *stdout* is mostly line-buffered and characters are not displayed on the screen until a '\n' is encountered. Other options are to use *fflush(stdout)* after the *printf* statement or *fprintf(stderr, fmt, ...)* instead of *printf*, since *stderr* is unbuffered.

For one more example, let's consider string functions. There is a good chance for string functions to cause an abrupt segmentation fault either because the given string pointer was NULL or because the string lacked the null character at the end. The following code will let you know what happened:

```
printf ("String pointer = %p\n", str);
printf ("length = %d\n", strlen (str));
// actual operation with str
```

The first *printf* will let you know if you are dealing with a NULL pointer. If the second one gives you no output or something insane, you can be sure that the string lacks a null character at the end (a short-cut instead of running a loop to check each character).

Conditional compiling

Unlike the *printf* lines that you add when there is an issue and remove once it is fixed, you might want some logging code to be always there, active throughout the development/maintenance cycle. Adding it before each coding cycle and removing it just before the production build is a horrible idea, and that's where you can rely on the power of the C preprocessor.

The basic concept is to wrap the debug code inside a *#if* block that checks for a macro, which is only defined for debug builds.

```
// production code
#ifdef MYDEBUG
    // logging code
#endif
// production code
```

You can compile this using *gcc* with the option *-DMYDEBUG* (which defines the macro MYDEBUG) during development in order to get the logging code compiled.

A cleaner approach would be to create a header file *debug.h* as follows:

```
#ifndef _MYDEBUG_H
#define _MYDEBUG_H

#ifdef MYDEBUG
    #include <stdio.h>
    #define log_printf(...) printf(__VA_ARGS__)
#else
    #define log_printf
#endif
#endif
```

It still requires you to compile with *-DMYDEBUG*, but the beauty is that wherever you include this header file, you can use the function *log_printf* without additional checking, which will vanish magically if it is a production build. Let's look at the example below:

```
#include <stdio.h>
#include "debug.h"

int main() {
    puts ("This is production code");
    log_printf ("This is debug log that will vanish if you
don't compile with -DMYDEBUG\n");

    return 0;
}
```

If you compile this without *-DMYDEBUG*, you won't get the *log_printf* message.